Global Navigation Satellite Systems

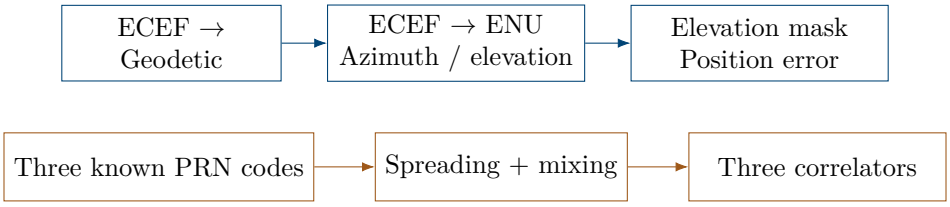
Exercise 4

Coordinate Frames, Satellite Visibility, and PRN Correlation

Winter Semester 2026/27

The engineering storyline

A GNSS receiver first needs to interpret satellite and receiver positions in suitable coordinate frames. It then determines which satellites are visible and whether excluding low-elevation signals helps positioning. Finally, it must distinguish several simultaneous satellite signals using their known spreading codes.



Timing: Problem 1: 15 min Problem 2: 20 min Problem 3: 25 min Problem 4: 25 min Discussion: 5 min. Homework is separate.

0. Before you begin

Keep the following files in the same folder:

File	What it contains
`Exercise_04.pdf`	Problems, equations, milestones, and output requirements.
`exercise_04.py`	Python starter with marked `TODO` sections. **Edit this file.**
`gnss_helpers.py`	Provided data loader, sky-plot utility, and positioning simulation. **Do not modify.**
`gnss_ex04_data.npz`	Receiver positions, broadcast-derived satellite positions, satellite angles, and three GPS L1 C/A PRN codes.
`requirements.txt`	Required Python packages (NumPy and Matplotlib).

Install the packages if needed: `python -m pip install -r requirements.txt`.

Run *one problem at a time* using `python exercise_04.py 1`, `2`, `3`, or `4`. The command automatically calls `task1()`, `task2()`, etc. Each problem contains its own input data and can be attempted independently.

Problem 1 – ECEF to Geodetic Coordinates

15 minutes

Situation. You know the receiver's position as three Cartesian ECEF coordinates. Convert this position into latitude, longitude, and height on the WGS84 ellipsoid, as discussed in Lecture 3.

Provided. The receiver position and WGS84 parameters are already loaded.

Python variable	Meaning	Shape / unit
`r_ecef_m`	Receiver ECEF position $\mathbf{r} = [x, y, z]^\mathsf{T}$	`(3,)` / m
`A_E`	WGS84 semi-major axis $a_E = 6378137$	scalar / m
`F`	WGS84 flattening $f = 1/298.257223563$	scalar / –
`E2`	First eccentricity squared $e_E^2 = 2f - f^2$	scalar / –

A. Convert Cartesian coordinates to longitude

Compute the distance p from Earth's rotation axis and the longitude λ:

$$p = \sqrt{x^2 + y^2}, \qquad \lambda = \operatorname{atan2}(y, x).$$

Implement. Inside `ecef_to_geodetic()`, calculate `p` and `lam` using NumPy. Angles returned by NumPy are in radians. **Required output.** The intermediate values `p` [m] and `lam` [rad] are available for the next milestone.

B. Iterate to obtain geodetic latitude

Start with $\phi^{(0)} = \operatorname{atan2}(z, p(1 - e_E^2))$ and repeat

$$N(\phi) = \frac{a_E}{\sqrt{1 - e_E^2 \sin^2 \phi}}, \quad \phi^{(k+1)} = \operatorname{atan2}(z + e_E^2 N(\phi^{(k)}) \sin \phi^{(k)}, p).$$

$N(\phi)$ is the prime-vertical radius of curvature [m], and k is the iteration number. **Implement.** Update `phi` until the change is smaller than 10^{-12} rad (maximum 30 iterations). Use `np.sin`, `np.sqrt`, and `np.arctan2`. **Required output.** Converged geodetic latitude `phi` [rad].

C. Calculate ellipsoidal height and return the coordinates

$$h = \frac{p}{\cos \phi} - N(\phi).$$

Implement. Recalculate N with the final ϕ. Return `(np.degrees(phi), np.degrees(lam), h)`. Replace the `raise NotImplementedError` line. **Required output.** Latitude [deg], longitude [deg], ellipsoidal height [m].

Execution. Only edit `ecef_to_geodetic()`; the given `task1()` loads the receiver, calls your function, and prints the result. **Run:** `python exercise_04.py 1`

Interpretation. Why is $\operatorname{atan2}(z, p)$ not generally the *geodetic* latitude? Is the computed height referenced to mean sea level or to WGS84?

Problem 2 – Satellite Viewing Directions and Sky Plot

20 minutes

Situation. Satellite ECEF positions do not directly tell us where to look in the local sky. Convert receiver-to-satellite displacement vectors to the East–North–Up (ENU) coordinate frame, then calculate azimuth and elevation.

Provided. Problem 2 loads its own receiver and satellite data; no results from Problem 1 are needed.

Python variable	Meaning	Shape / unit
`rec`	Receiver ECEF position $\mathbf{r}$	`(3,)` / m
`phi, lam, h`	Receiver geodetic coordinates	3 scalars / deg, deg, m
`sats`	Satellite positions $\mathbf{s}_i$ in ECEF	`(10,3)` / m
`labels`	Galileo satellite identifiers	`(10,)` / strings

A. ECEF displacement to local ENU

For satellite i, form its line-of-sight displacement $\mathbf{d}_i = \mathbf{s}_i - \mathbf{r}$, then rotate:

$$\underbrace{\begin{bmatrix} E_i \\ N_i \\ U_i \end{bmatrix}}_{\mathbf{d}_{\mathrm{ENU},i}} = \underbrace{\begin{bmatrix} -\sin\lambda & \cos\lambda & 0 \\ -\sin\phi\cos\lambda & -\sin\phi\sin\lambda & \cos\phi \\ \cos\phi\cos\lambda & \cos\phi\sin\lambda & \sin\phi \end{bmatrix}}_{\mathbf{R}_{\mathrm{ENU}}(\phi,\lambda)} \mathbf{d}_i.$$

E_i, N_i, U_i are the east, north, and up components [m]. Use radians in trigonometric expressions. **Implement.** Complete `ecef_to_enu(sat_ecef_m, rec_ecef_m, phi_deg, lam_deg)` by constructing the 3×3 matrix with NumPy and returning `R @ d`. **Required output.** For each satellite: a 3-element ENU displacement vector [m].

B. ENU to azimuth and elevation

Azimuth A_i is measured clockwise from north; elevation ε_i is measured above the local horizon.

$$A_i = \Big[\operatorname{atan2}(E_i, N_i)\Big]_{0^\circ\ldots360^\circ}, \qquad \varepsilon_i = \operatorname{atan2}\left(U_i, \sqrt{E_i^2 + N_i^2}\right).$$

Implement. Complete `enu_to_angles(enu_m)`. Convert both results to degrees, and apply `% 360` to azimuth. **Required output.** Two scalar angles per satellite: azimuth [deg] and elevation [deg].

Problem 2 – Satellite Viewing Directions and Sky Plot (continued)

C. Process all satellites

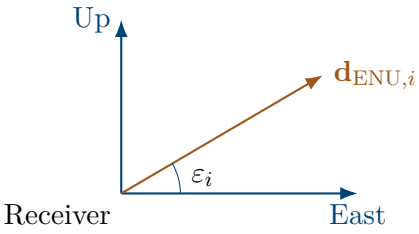
Implement. Complete the loop in `task2()`. For each satellite, call `ecef_to_enu()` followed by `enu_to_angles()`. Store the angles in `az_deg` and `el_deg`; print one line per satellite. **Required output.** A table containing 10 satellite identifiers, 10 azimuths, and 10 elevations.

D. Visualize satellite visibility

Provided. The plotting function `skyplot(az_deg, el_deg, labels, mask_deg=15)` is implemented in `gnss_helpers.py`. **Implement.** Call `skyplot(...,mask_deg=15)` using your arrays and then `plt.show()`. No plotting function needs to be implemented. **Required output.** A polar sky plot: north at top, azimuth clockwise, zenith at center, and a dashed 15° mask circle.

Execution. Edit `ecef_to_enu()`, `enu_to_angles()`, and the loop in `task2()`. **Run:** `python exercise_04.py 2`. This calls `task2()`, which uses your two new functions and the provided `skyplot()` helper.

Sanity check. Galileo E27 should appear near azimuth 284.5° and elevation 77.2°.



Elevation is an angle in a vertical section.
Azimuth is measured in the horizontal plane.

Interpretation. Why is a high-elevation satellite close to the center of the sky plot? Which satellites fall below the illustrated 15° mask?

Problem 3 – Elevation Masking and Position Quality

25 minutes

Situation. Low-elevation measurements may contain larger errors, but discarding satellites also reduces the measurements available to estimate receiver position and clock bias. Investigate both effects.

Provided. Problem 3 is independent of Problem 2. The following arrays and parameters are already loaded in `task3()`.

Python variable	Meaning	Shape / unit
`sats`	Satellite ECEF positions	`(10,3)` / m
`rec`	Receiver ECEF position	`(3,)` / m
`el`	Satellite elevation angles	`(10,)` / deg
`labels`	Satellite identifiers	`(10,)` / strings
`masks`	Test mask angles $0, 10, 20, 30$	4 values / deg

A. Select satellites by elevation

For each mask $\varepsilon_{\min}$, retain satellite i only if

$$\varepsilon_i \geq \varepsilon_{\min}.$$

Implement. In `task3()`, write a loop over `masks`. Use `keep = el >= mask`, `labels[keep]`, and `np.sum(keep)` to select, identify, and count the remaining satellites. **Required output.** For each mask: mask angle, number of retained satellites, and their IDs.

B. Run two controlled positioning simulations

Provided. The function `mask_experiment()` is complete. It generates synthetic pseudoranges, fits receiver ECEF position and common receiver-clock range bias, and repeats the calculation over 200 trials. You do *not* implement least squares. **Implement.** Call `mask_experiment()` twice with the provided positions, elevations and masks: first with `'constant'`, then with `'elevation'`. Store both result arrays and print their values. **Required output.** Two arrays of shape `(4,4)`. Each row contains: mask [deg], retained count [–], 3D position RMSE [m], 95th-percentile 3D error [m].

How the simulation measures accuracy. For trial k, compare the estimated position $\hat{\mathbf{r}}^{(k)}$ to the known receiver position $\mathbf{r}$:

$$\mathrm{RMSE} = \sqrt{\frac{1}{200}\sum_{k=1}^{200} \left\|\hat{\mathbf{r}}^{(k)} - \mathbf{r}\right\|^2}.$$

Here RMSE is the root-mean-square 3D error across the 200 simulated trials.

Problem 3 – Elevation Masking and Position Quality (continued)

C. Plot and compare the two scenarios

The two illustrative models are defined inside the provided simulation:

Scenario	Assumed pseudorange measurement errors
Equal-quality	Noise standard deviation $\sigma_i = 3$ m and bias $\beta_i = 0$ for every satellite.
Elevation-dependent	$\sigma_i = 2 + 20e^{-\varepsilon_i/12^\circ}$ m and $\beta_i = 12e^{-\varepsilon_i/10^\circ}$ m.

σ_i is the random error standard deviation and β_i is an additional positive error bias; ε_i is expressed in degrees in these illustrative exponential laws. The model is pedagogical, not measured receiver performance.

Provided. The helper `plot_mask_results(results, ax=ax, label=...)` plots mask angle against 3D position RMSE and annotates the number of satellites. **Implement.** Create a Matplotlib figure and axes, call `plot_mask_results()` twice on the *same axes*, then `plt.show()`. Use meaningful labels for both scenarios. **Required output.** One comparison plot showing both RMSE curves and retained-satellite counts at 0°, 10°, 20°, and 30°.

Execution. Edit only `task3()`. The functions `mask_experiment()` and `plot_mask_results()` are given. **Run:** `python exercise_04.py 3`. This calls `task3()` and executes Parts A–C in order.

Interpretation. Answer the following briefly:

a) Does a higher elevation mask always improve positioning accuracy?

b) Why does the constant-quality scenario behave differently from the elevation-dependent scenario?

c) At 30°, how many satellites remain? Why might this be risky even though a solution is mathematically possible?

Model limitation. The estimator is an unweighted, four-unknown pseudorange least-squares model with common clock range bias b [m]. The satellite ECEF positions are from Galileo broadcast ephemerides, but the pseudorange errors are synthetic. No ionospheric, tropospheric, satellite-clock, or Earth-rotation propagation corrections are evaluated.

Problem 4 – Separating Simultaneous Satellite Data with PRN Codes

25 minutes

Situation. Three satellites transmit simultaneously using different PRN spreading codes. Their received signals overlap and contain noise. Recover one data symbol from each satellite using correlation.

Provided. The three GPS L1 C/A code sequences are supplied, not generated. The data symbols and relative signal amplitudes are illustrative.

Python variable	Meaning	Shape / value
`codes`	Three distinct GPS L1 C/A PRN sequences $c_i[n]$	`(3,1023)` / ± 1
`symbols`	One data symbol d_i per satellite	`(3,)` / $[+1, -1, +1]$
`amps`	Positive relative amplitudes a_i	`(3,)` / $[1, .8, .65]$
`noise`	Independent Gaussian samples $w[n]$	`(1023,)` / $\sigma_w = 4$

Simplified baseband model: exactly one bipolar data symbol is spread over one 1023-chip code period. The three codes are perfectly aligned; no code delay, carrier Doppler, or carrier phase is modeled. In real GPS L1 C/A, a 20 ms navigation bit spans 20 repetitions of the 1 ms code.

A. Spread each data symbol

$$x_i[n] = a_i\, d_i\, c_i[n], \qquad i = 1, 2, 3, \qquad n = 0, \ldots, 1022.$$

Implement. Inside `task4()`, create `spread` with shape `(3,1023)` and fill each row using the supplied `amps`, `symbols`, and `codes`. A short `for` loop is sufficient. **Required output.** Three spread waveforms, one per satellite. Print `spread.shape` as a size check.

B. Combine the satellites and add noise

$$u[n] = \sum_{i=1}^{3} x_i[n], \qquad y[n] = u[n] + w[n].$$

Implement. Compute `combined = np.sum(spread, axis=0)` and `y = combined + noise`. Plot the *first 40 samples* of `combined` and `y` in two separate clearly labeled figures. **Required output.** Two vectors of shape `(1023,)` and two plots: *before noise* and *after noise*. Use chip index on the horizontal axis.

Problem 4 – Separating Simultaneous Satellite Data (continued)

C. Implement the three-code correlator bank

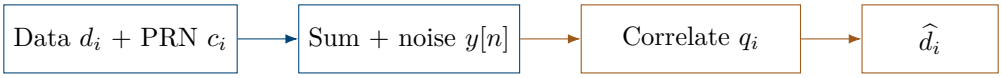
Each correlator uses the *same* received mixture $y[n]$, but a different known code $c_i[n]$. Calculate

$$q_i = \frac{1}{1023} \sum_{n=0}^{1022} y[n]c_i[n], \qquad \widehat{d}_i = \text{sign}(q_i).$$

Here q_i is the normalized correlator output, and $\widehat{d}_i$ is the recovered data symbol. Since each $a_i > 0$, the sign of the score represents the data symbol. The ideal contribution is $a_i d_i$; interference and noise cause deviations.

Implement. Create `scores = np.zeros(3)`. For each satellite, compute `np.mean(y * codes[i, :])`. Apply `np.sign(scores)` to obtain the three recovered symbols and print both arrays beside `symbols`. **Required output.** Three signed correlation scores and three estimated data symbols. Compare recovered and transmitted symbols. **No correlation bar chart is required.**

Execution. Edit only `task4()`. The code sequences and additive noise generation are provided. **Run:** `python exercise_04.py 4`. This calls `task4()` and runs Parts A–C.



Interpretation. Why is the composite waveform difficult to interpret by eye? Why do the three correlators recover different symbols from the same $y[n]$? What does a negative correlation score indicate?

Helpful observation. The data symbols are ± 1, not binary 0 and 1 at this processing stage. In this exercise the receiver knows the PRN codes and their alignment, so no code acquisition is required.

Homework – Elevation-Dependent Pseudorange Errors

Engineering question. The positioning simulation in Problem 3 treated low-elevation measurements as less reliable. Now implement the corresponding measurement-error model yourself and observe how the masking results change when its parameters change.

Provided. The same ten satellite positions and elevations as Problem 3. The positioning simulation is provided; it can accept a custom function that returns two arrays of measurement errors.

Variable	Meaning	Shape / unit
`el_deg`	Elevation angles input to the model	`(10,)` / deg
`sigma_m`	Random pseudorange error standard deviations	`(10,)` / m
`bias_m`	Systematic positive pseudorange errors	`(10,)` / m

A. Implement the elevation error model

$$\sigma_i = 2 + 20\exp(-\varepsilon_i/12^\circ)\ \text{m}, \qquad \beta_i = 12\exp(-\varepsilon_i/10^\circ)\ \text{m}.$$

Implement. Complete `elevation_error_model(el_deg)` using NumPy arrays. Return `(sigma_m, bias_m)` with the same shape as the elevation input. The exponential uses elevations numerically in degrees. **Required output.** For each satellite: noise standard deviation σ_i [m] and bias β_i [m].

B. Check the behavior at three elevations

Implement. In `homework()`, call your model at 10°, 30°, and 60°. Print both parameters in a small table. **Required output.** Three rows with elevation [deg], σ_i [m], and β_i [m].

C. Connect your model to the positioning simulation

```
custom = mask_experiment(sats, rec, el,
    mask_values=(0,10,20,30),
    error_model=elevation_error_model)
```

Implement. Run this experiment and the built-in elevation-dependent scenario from Problem 3. Compare their RMSE values; they should agree to numerical precision. **Required output.** An RMSE comparison table for all four elevation masks and the maximum difference [m].

D. Investigate a parameter change

Implement. Create a second model with the coefficient 20 reduced to 10 in σ_i (leave β_i unchanged). Run the simulation once more and plot both RMSE curves using `plot_mask_results()`. **Required output.** One plot comparing the original and modified models, with RMSE [m] versus elevation mask [deg].

Interpretation. Where is random noise largest? Why can reducing low-elevation noise change the preferred elevation mask? Is the best mask universal?

Run: `python exercise_04.py hw`. This calls `homework()`, which in turn uses your completed error-model function.

Submission: Completed homework functions in `exercise_04.py`; the printed table, one comparison plot, and a short interpretation (3–5 sentences).